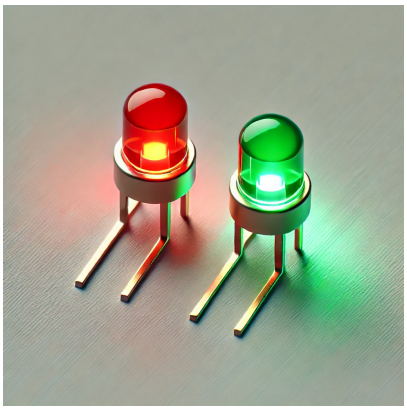
Diferents Programes amb Dos LEDs

Aquest document recull diferents implementacions per al control de dos LEDs



Índex

Dos LEDs amb el mateix ritme i time.sleep
Dos LEDs amb el mateix ritme, control amb time.sleep i neteja amb try-except
Dos LEDs alternats amb funcions modulars
Dos LEDs alternats amb funcions modulars i neteja automàtica
Dos LEDs amb intervals diferents i time slicing
Dos LEDs amb intervals diferents i threading
Dos LEDs amb intervals diferents i multiprocessing

Dos LEDs amb el mateix ritme i time.sleep

Codi:

```
import RPi.GPIO as IO
import time

pinled1 = 4
pinled2 = 17
pausa = 1

IO.setmode(IO.BCM)
IO.setup(pinled1, IO.OUT)
IO.setup(pinled2, IO.OUT)

while True:
    IO.output(pinled1, IO.HIGH)
    IO.output(pinled2, IO.HIGH)
    time.sleep(pausa)

    IO.output(pinled1, IO.LOW)
    IO.output(pinled2, IO.LOW)
    time.sleep(pausa)
```

Explicació línia a línia:

1. import RPi.GPIO as IO: Importa el mòdul RPi.GPIO i el renombra com 'IO' per controlar els pins GPIO de la Raspberry Pi.

2. import time: Importa el mòdul 'time' per gestionar els retards en l'execució del codi.

3. pinled1 = 4: Assigna el pin GPIO 4 per al primer LED.

4. pinled2 = 17: Assigna el pin GPIO 17 per al segon LED.

5. pausa = 1: Defineix un temps de pausa d'1 segon entre els estats ON i OFF dels LEDs.

6. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM, utilitzant la numeració de pines de la Raspberry Pi.

7. IO.setup(pinled1, IO.OUT): Configura el pin GPIO 4 com a sortida per controlar el primer LED.

8. IO.setup(pinled2, IO.OUT): Configura el pin GPIO 17 com a sortida per controlar el segon LED.

9. while True:: Comença un bucle infinit per alternar l'estat dels LEDs.

10. IO.output(pinled1, IO.HIGH): Activa el primer LED.

11. IO.output(pinled2, IO.HIGH): Activa el segon LED.

12. time.sleep(pausa): Espera durant 1 segon abans de canviar l'estat dels LEDs.

13. IO.output(pinled1, IO.LOW): Apaga el primer LED.

14. IO.output(pinled2, IO.LOW): Apaga el segon LED.

15. time.sleep(pausa): Espera durant 1 segon abans de repetir el cicle.

Dos LEDs amb el mateix ritme, control amb time.sleep i neteja amb try-except

Codi:

```
import RPi.GPIO as IO
import time

pinled1 = 4
pinled2 = 17
pausa = 1

IO.setmode(IO.BCM)
IO.setup(pinled1, IO.OUT)
IO.setup(pinled2, IO.OUT)

try:
    while True:
        IO.output(pinled1, IO.HIGH)
        IO.output(pinled2, IO.HIGH)
        time.sleep(pausa)

        IO.output(pinled1, IO.LOW)
        IO.output(pinled2, IO.LOW)
        time.sleep(pausa)

except KeyboardInterrupt:
    IO.cleanup()
```

Explicació línia a línia:

1. import RPi.GPIO as IO: Importa el mòdul RPi.GPIO i el renombra com 'IO' per controlar els pins GPIO de la Raspberry Pi.

2. import time: Importa el mòdul 'time' per gestionar els retards en l'execució del codi.

3. pinled1 = 4: Assigna el pin GPIO 4 per al primer LED.

4. pinled2 = 17: Assigna el pin GPIO 17 per al segon LED.

5. pausa = 1: Defineix un temps de pausa d'1 segon entre els estats ON i OFF dels LEDs.

6. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM, utilitzant la numeració de pines de la Raspberry Pi.

7. IO.setup(pinled1, IO.OUT): Configura el pin GPIO 4 com a sortida per controlar el primer LED.

8. IO.setup(pinled2, IO.OUT): Configura el pin GPIO 17 com a sortida per controlar el segon LED.

9. try: Inicia un bloc 'try' per capturar excepcions durant l'execució del bucle.

10. while True:: Comença un bucle infinit per alternar l'estat dels LEDs.

11. IO.output(pinled1, IO.HIGH): Activa el primer LED.

12. IO.output(pinled2, IO.HIGH): Activa el segon LED.

13. time.sleep(pausa): Espera durant 1 segon abans de canviar l'estat dels LEDs.

14. IO.output(pinled1, IO.LOW): Apaga el primer LED.

15. IO.output(pinled2, IO.LOW): Apaga el segon LED.

16. time.sleep(pausa): Espera durant 1 segon abans de repetir el cicle.

17. except KeyboardInterrupt:: Captura una interrupció de teclat per aturar el codi de manera segura.

18. IO.cleanup(): Allibera els recursos GPIO per evitar errors futurs.

Dos LEDs alternats amb funcions modulars

Codi:

```
import RPi.GPIO as IO
import time

def setup_gpio():
    IO.setmode(IO.BCM)
    IO.setup(pinled1, IO.OUT)
    IO.setup(pinled2, IO.OUT)

def blink_leds():
    IO.output(pinled1, IO.HIGH)
    IO.output(pinled2, IO.LOW)
    time.sleep(pausa)
    IO.output(pinled1, IO.LOW)
    IO.output(pinled2, IO.HIGH)
    time.sleep(pausa)

def netejaIO():
    IO.cleanup()

pinled1 = 4
pinled2 = 17
pausa = 1

setup_gpio()

try:
    while True:
        blink_leds()
except KeyboardInterrupt:
```

```
    netejaIO()
```

Explicació línia a línia:

1. import RPi.GPIO as IO: Importa el mòdul RPi.GPIO i el renombra com 'IO' per controlar els pins GPIO de la Raspberry Pi.

2. import time: Importa el mòdul 'time' per gestionar els retards en l'execució del codi.

3. def setup_gpio(): Defineix una funció per configurar els pins GPIO.

4. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM.

5. IO.setup(pinled1, IO.OUT): Configura el pin GPIO 4 com a sortida per al primer LED.

6. IO.setup(pinled2, IO.OUT): Configura el pin GPIO 17 com a sortida per al segon LED.

7. def blink_leds(): Defineix una funció per alternar l'encesa dels LEDs.

8. IO.output(pinled1, IO.HIGH): Activa el primer LED.

9. IO.output(pinled2, IO.LOW): Apaga el segon LED.

10. time.sleep(pausa): Espera el temps definit abans de canviar l'estat dels LEDs.

11. IO.output(pinled1, IO.LOW): Apaga el primer LED.

12. IO.output(pinled2, IO.HIGH): Activa el segon LED.

13. def netejaIO(): Defineix una funció per alliberar els recursos GPIO.

14. pinled1 = 4, pinled2 = 17: Assigna els pins GPIO per als LEDs.

15. pausa = 1: Defineix un temps de pausa entre els estats dels LEDs.

16. setup_gpio(): Crida la funció per configurar els pins.

17. try: Inicia un bloc 'try' per gestionar la interrupció del teclat.

18. while True:: Comença un bucle infinit per alternar l'estat dels LEDs.

19. blink_leds(): Crida la funció per alternar l'encesa dels LEDs.

20. except KeyboardInterrupt:: Captura una interrupció de teclat per aturar el codi de manera segura.

21. netejaIO(): Allibera els recursos GPIO per evitar errors futurs.

Dos LEDs alternats amb funcions modulars i neteja automàtica

Codi:

```
import RPi.GPIO as IO
import time

def setup_gpio():
    IO.setmode(IO.BCM)
    IO.setup(pinled1, IO.OUT)
    IO.setup(pinled2, IO.OUT)

def blink_leds():
    IO.output(pinled1, IO.HIGH)
    IO.output(pinled2, IO.LOW)
    time.sleep(pausa)
    IO.output(pinled1, IO.LOW)
    IO.output(pinled2, IO.HIGH)
    time.sleep(pausa)

def netejaIO():
    IO.cleanup()

pinled1 = 4
pinled2 = 17
pausa = 1

setup_gpio()

try:
    while True:
        blink_leds()
except KeyboardInterrupt:
```

```
    netejaIO()
```

Explicació línia a línia:

1. import RPi.GPIO as IO: Importa el mòdul RPi.GPIO i el renombra com 'IO' per controlar els pins GPIO de la Raspberry Pi.

2. import time: Importa el mòdul 'time' per gestionar els retards en l'execució del codi.

3. def setup_gpio(): Defineix una funció per configurar els pins GPIO.

4. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM.

5. IO.setup(pinled1, IO.OUT): Configura el pin GPIO 4 com a sortida per al primer LED.

6. IO.setup(pinled2, IO.OUT): Configura el pin GPIO 17 com a sortida per al segon LED.

7. def blink_leds(): Defineix una funció per alternar l'encesa dels LEDs.

8. IO.output(pinled1, IO.HIGH): Activa el primer LED.

9. IO.output(pinled2, IO.LOW): Apaga el segon LED.

10. time.sleep(pausa): Espera el temps definit abans de canviar l'estat dels LEDs.

11. IO.output(pinled1, IO.LOW): Apaga el primer LED.

12. IO.output(pinled2, IO.HIGH): Activa el segon LED.

13. def netejaIO(): Defineix una funció per alliberar els recursos GPIO.

14. pinled1 = 4, pinled2 = 17: Assigna els pins GPIO per als LEDs.

15. pausa = 1: Defineix un temps de pausa entre els estats dels LEDs.

16. setup_gpio(): Crida la funció per configurar els pins.

17. try: Inicia un bloc 'try' per gestionar la interrupció del teclat.

18. while True:: Comença un bucle infinit per alternar l'estat dels LEDs.

19. blink_leds(): Crida la funció per alternar l'encesa dels LEDs.

20. except KeyboardInterrupt:: Captura una interrupció de teclat per aturar el codi de manera segura.

21. netejaIO(): Allibera els recursos GPIO per evitar errors futurs.

Dos LEDs amb intervals diferents i time slicing

Codi:

```
import RPi.GPIO as IO
import time

pin_led1 = 4
pin_led2 = 17

interval_led1 = 5
interval_led2 = 1

IO.setmode(IO.BCM)
IO.setup(pin_led1, IO.OUT)
IO.setup(pin_led2, IO.OUT)

last_time_led1 = time.time()
last_time_led2 = time.time()

try:
    while True:
        current_time = time.time()

        if current_time - last_time_led1 >= interval_led1:
            IO.output(pin_led1, not IO.input(pin_led1))
            last_time_led1 = current_time

        if current_time - last_time_led2 >= interval_led2:
            IO.output(pin_led2, not IO.input(pin_led2))
            last_time_led2 = current_time

        time.sleep(0.01)
```

```
except KeyboardInterrupt:
    IO.cleanup()
```

Explicació línia a línia:

1. import RPi.GPIO as IO: Importa el mòdul RPi.GPIO i el renombra com 'IO' per controlar els pins GPIO de la Raspberry Pi.

2. import time: Importa el mòdul 'time' per gestionar els intervals de temps.

3. pin_led1 = 4, pin_led2 = 17: Assigna els pins GPIO 4 i 17 per als LEDs.

4. interval_led1 = 5, interval_led2 = 1: Defineix intervals diferents per al LED 1 i LED 2 (5 i 1 segons respectivament).

5. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM, utilitzant la numeració de pines de la Raspberry Pi.

6. IO.setup(pin_led1, IO.OUT), IO.setup(pin_led2, IO.OUT): Configura els pins dels LEDs com a sortida.

7. last_time_led1 = time.time(), last_time_led2 = time.time(): Guarda el temps inicial per al control dels LEDs.

8. while True:: Comença un bucle infinit per gestionar els intervals dels LEDs.

9. current_time = time.time(): Obté el temps actual per verificar els intervals de cada LED.

10. if current_time - last_time_led1 >= interval_led1:: Verifica si ha passat l'interval de 5 segons per al LED 1.

11. IO.output(pin_led1, not IO.input(pin_led1)): Canvia l'estat del LED 1 a ON o OFF segons l'estat actual.

12. last_time_led1 = current_time: Actualitza el temps d'últim canvi d'estat per al LED 1.

13. if current_time - last_time_led2 >= interval_led2:: Verifica si ha passat l'interval d'1 segon per al LED 2.

14. IO.output(pin_led2, not IO.input(pin_led2)): Canvia l'estat del LED 2 a ON o OFF segons l'estat actual.

15. last_time_led2 = current_time: Actualitza el temps d'últim canvi d'estat per al LED 2.

16. time.sleep(0.01): Afegeix un petit retard per evitar la sobrecàrrega de la CPU.

17. except KeyboardInterrupt:: Captura una interrupció de teclat per aturar el codi de manera segura.

18. IO.cleanup(): Allibera els recursos GPIO per evitar errors futurs.

Dos LEDs amb intervals diferents i threading

Codi:

```
import RPi.GPIO as IO
import time
import threading

pin_led1 = 4
pin_led2 = 17

IO.setmode(IO.BCM)
IO.setup(pin_led1, IO.OUT)
IO.setup(pin_led2, IO.OUT)

def led1_blink():
    while True:
        IO.output(pin_led1, not IO.input(pin_led1))
        time.sleep(5)

def led2_blink():
    while True:
        IO.output(pin_led2, not IO.input(pin_led2))
        time.sleep(1)

if __name__ == '__main__':
    fil_1 = threading.Thread(target=led1_blink)
    fil_2 = threading.Thread(target=led2_blink)

    fil_1.start()
    fil_2.start()

    fil_1.join()
```

```
    fil_2.join()
```

Explicació línia a línia:

1. import RPi.GPIO as IO, import time, import threading: Importa els mòduls necessaris per gestionar els LEDs i els fils.

2. pin_led1 = 4, pin_led2 = 17: Assigna els pins GPIO per als LEDs.

3. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM per numerar els pines.

4. IO.setup(pin_led1, IO.OUT), IO.setup(pin_led2, IO.OUT): Configura els pins com a sortides per controlar els LEDs.

5. def led1_blink(): Defineix una funció per alternar l'estat del LED 1 cada 5 segons.

6. while True:: Inicia un bucle infinit dins de led1_blink per alternar el LED.

7. IO.output(pin_led1, not IO.input(pin_led1)): Canvia l'estat del LED 1.

8. time.sleep(5): Pausa de 5 segons abans de canviar l'estat del LED 1 novament.

9. def led2_blink(): Defineix una funció per alternar l'estat del LED 2 cada 1 segon.

10. IO.output(pin_led2, not IO.input(pin_led2)): Canvia l'estat del LED 2.

11. time.sleep(1): Pausa d'1 segon abans de canviar l'estat del LED 2 novament.

12. fil_1 = threading.Thread(target=led1_blink), fil_2 = threading.Thread(target=led2_blink): Crea fils per executar les funcions led1_blink i led2_blink.

13. fil_1.start(), fil_2.start(): Inicia els fils per executar les funcions en paral·lel.

14. fil_1.join(), fil_2.join(): Manté els fils actius fins que acabin.

Dos LEDs amb intervals diferents i multiprocessing

Codi:

```
import RPi.GPIO as IO
import time
import multiprocessing

pin_led1 = 4
pin_led2 = 17

IO.setmode(IO.BCM)
IO.setup(pin_led1, IO.OUT)
IO.setup(pin_led2, IO.OUT)

def led1_blink():
    while True:
        IO.output(pin_led1, not IO.input(pin_led1))
        time.sleep(5)

def led2_blink():
    while True:
        IO.output(pin_led2, not IO.input(pin_led2))
        time.sleep(1)

if __name__ == '__main__':
    process_1 = multiprocessing.Process(target=led1_blink)
    process_2 = multiprocessing.Process(target=led2_blink)

    process_1.start()
    process_2.start()

    process_1.join()
```

```
    process_2.join()
```

Explicació línia a línia:

1. import RPi.GPIO as IO, import time, import multiprocessing: Importa els mòduls necessaris per gestionar els LEDs i els processos.

2. pin_led1 = 4, pin_led2 = 17: Assigna els pins GPIO per als LEDs.

3. IO.setmode(IO.BCM): Configura el mòdul GPIO en el mode BCM per numerar els pines.

4. IO.setup(pin_led1, IO.OUT), IO.setup(pin_led2, IO.OUT): Configura els pins com a sortides per controlar els LEDs.

5. def led1_blink(): Defineix una funció per alternar l'estat del LED 1 cada 5 segons.

6. while True:: Inicia un bucle infinit dins de led1_blink per alternar el LED.

7. IO.output(pin_led1, not IO.input(pin_led1)): Canvia l'estat del LED 1.

8. time.sleep(5): Pausa de 5 segons abans de canviar l'estat del LED 1 novament.

9. def led2_blink(): Defineix una funció per alternar l'estat del LED 2 cada 1 segon.

10. IO.output(pin_led2, not IO.input(pin_led2)): Canvia l'estat del LED 2.

11. time.sleep(1): Pausa d'1 segon abans de canviar l'estat del LED 2 novament.

12. process_1 = multiprocessing.Process(target=led1_blink), process_2 = multiprocessing.Process(target=led2_blink): Crea processos per executar les funcions led1_blink i led2_blink.

13. process_1.start(), process_2.start(): Inicia els processos per executar les funcions en paral·lel.

14. process_1.join(), process_2.join(): Manté els processos actius fins que acabin.